TECMILENIO

Máster:
Inteligencia Artificial

Tetramestre:
Septiembre-diciembre 2026

Materia:
Gestión de Proyectos de inteligencia Artificial

Proyecto Fase II · Contenedor Docker: instrucciones de ejecución

Matrícula y nombre del estudiante:
T07175241 - Gabriel Alejandro García Ángeles

Fecha: 01/11/2026

Contenido de esta entrega

La solución se ejecuta con dos contenedores (frontend y API) o con una sola imagen, que es la que se desplegaría en la nube. Los archivos de contenerización están en la carpeta Docker de este ZIP y en el repositorio https://github.com/gagarang25/ProyectoFase2 (commit 70fa349, etiqueta app-v1.0.0).

Tabla 1: Archivos de contenerización.

Archivo	Para qué sirve
Dockerfile	Imagen de la API: Python 3.12, PyTorch CPU, router mDeBERTa-v3 (model-v1.0.0), modelos de ETA y riesgo, SAP simulado, pruebas y chat estático
frontend/Dockerfile y frontend/nginx.conf	Imagen del chat web con nginx como proxy inverso hacia la API
docker-compose.yml	Levanta web (8080) y api (8000) y abre el chat cuando la API está sana
.env.example	Variables de configuración (fecha de referencia, umbral, hilos, CORS, bitácora)
.dockerignore	Excluye datos de entrenamiento, resultados y documentos de la imagen

1. Requisitos

- Docker Desktop 4.x (Windows o macOS) o Docker Engine 24+ con Compose v2 (Linux), con al menos 4 GB de memoria asignada.
- Unos 10 GB libres en disco e internet la primera vez (descarga PyTorch y el modelo de Hugging Face).
- Git y, solo para las pruebas funcionales, Python 3.10 o superior (sin librerías adicionales).

2. Ejecución con docker compose (dos contenedores)

```
git clone https://github.com/gagarang25/ProyectoFase2.git
cd ProyectoFase2
copy .env.example .env                  # Git Bash o Linux: cp .env.example .env
docker compose build --build-arg GIT_COMMIT=$(git rev-parse --short HEAD)
docker compose up -d
docker compose ps                       # api y web deben quedar (healthy)
```

- Chat: http://localhost:8080 (elegir un cliente arriba y tocar una orden abierta).
- Documentación de la API (Swagger): http://localhost:8080/docs
- Estado y versiones: http://localhost:8000/api/salud

La primera construcción tarda alrededor de 8 minutos; las siguientes, segundos. La API usa 1.7 a 1.9 GB de RAM.

3. Ejecución de la imagen única (simulación del despliegue en la nube)

```
docker build -t cab-chatbot-api:1.0.0 .      # si no se construyó con compose
docker run -d --name cab-nube -p 8001:8000 cab-chatbot-api:1.0.0
```

El chat y la API quedan juntos en http://localhost:8001. Es la misma imagen que se subiría a AWS Elastic Beanstalk, Google Cloud Run o Azure Container Apps (ver el manual de despliegue).

4. Pruebas

```
docker compose exec api pytest -q              # 71 pruebas dentro del contenedor
python pruebas/pruebas_funcionales.py           # 35 casos contra http://localhost:8080
python pruebas/pruebas_funcionales.py --url http://localhost:8001
```

Resultado en la validación (commit 70fa349): 71/71 en pytest, 35/35 casos en las dos modalidades y 160/160 respuestas del router iguales a las del entrenamiento.

5. Detener y limpiar

```
docker compose down
docker rm -f cab-nube
```

Si algo falla (puerto ocupado, memoria insuficiente, contenedor unhealthy), la tabla de solución de problemas está en la sección 9.2 del manual de despliegue.